МИНОБРНАУКИ РОССИИ
федеральное государственное бюджетное образовательное учреждение высшего образования
«САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ ЭКОНОМИЧЕСКИЙ УНИВЕРСИТЕТ»
(СПбГЭУ)

Факультет экономики, финансов и информационных технологий
Кафедра прикладной математики и экономико-математических методов

КУРСОВАЯ РАБОТА

по дисциплине:
«**Языки и методы программирования**»

Тема: «Алгоритмы генерации и проверки геометрических задач»

Направление 01.03.02 «Прикладная математика и информатика»

Направленность «Прикладная математика и информатика в экономике и управлении»

Обучающийся Москвитин Марк Александрович

Группа ПМ-2401 Подпись_____

Проверил: Леора Светлана Николаевна
Должность: доцент, к.ф.-м.н.

Оценка _____ Дата: «___» _____ 202_ г.

Подпись: _____

Санкт-Петербург
2025

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 3
1. ИССЛЕДОВАТЕЛЬСКАЯ ЧАСТЬ
1.1. Поле и поиск в глубину 5
1.2. Генерация заданий и адаптивная сложность 6
1.3 Адаптивная сложность 7
2. ПРАКТИЧЕСКАЯ ЧАСТЬ
2.1. Первоначальная отрисовка поля 9
2.2. Работа с полем 10
2.3. Работа с заданиями 12
2.4. Адаптивная сложность 17
2.5. Особенности интерфейса 18
3. ЭКОНОМИЧЕСКОЕ ОБОСНОВАНИЕ
3.1. Исследование потребителя 20
3.2. Анализ конкурентов 20
3.3. Затраты 21
3.4. Монетизация 23
3.5. Продажа лицензии 24
ВЫВОДЫ 25
ЗАКЛЮЧЕНИЕ 26
СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ 27

ВВЕДЕНИЕ

В современном мире люди очень много времени проводят в электронных устройствах. Тем более, само по себе, это времяпрепровождение бывает довольно бесполезно. Какую выгоду принесут долгие скроллинги ленты социальных сетей или победа в Королевской битве?

Да, возможно, эти действия приведут к выработке положительных эмоций от радости выигрыша или смешных роликов. Вместе с тем, исследования показывают, что при скроллинге человек испытывает стресс. Это также снижает концентрацию, память и самоконтроль. Вы постепенно начинаете привыкать к постоянной смене эмоций не только от чередования видео, но и от повседневных активностей.

Постепенно избавиться от вредных привычек вам могут помочь интерактивные головоломки. Вместо того, чтобы бездумно перелистывать контент за контентом, можно скачать любую подобную игру и тренировать свой мозг.

Одним из примеров таких головоломок являются геометрические задачи. В зависимости от трактовки этого понятия, на рынке можно найти различные их реализации. В данной работе под геометрическими задачами стоит понимать упражнения, для решения которых требуются базовые знания теоретической геометрии.

Euclidea и Пифагория – известные сборники подобных головоломок. Еще в школьном возрасте я пытался пройти как можно дальше. Еще тогда мне было интересно, как именно устроена эта система.

Однако в их реализации присутствует существенный минус: набор задач постоянен и ограничен, поэтому в какой-то момент пользователь дойдет до конца и удалит приложение из-за него ненадобности?

Да, конечно, можно с каждым разом повышать уровень сложности, но действия потребителя от этого не изменятся: разочаровавшись в своих способностях, он просто больше не будет пользоваться игрой. Создатели

Euclidea и Пифагории решили пойти именно этим путем, работая над сиквелами и дополнениями к оригинальным концепциям.

Целью данной курсовой работы является разработка решения, которое позволит бесконечно создавать геометрические задачи и не терять пользователей из-за ограниченности контента. Это позволит более продуктивно проводить время в электронных устройствах, постепенно уменьшая зависимость от вредных гаджетов и развлечений.

Задачи курсовой работы:

1) усовершенствование навыков программирования на языке Python [2] с использованием библиотеки Pygame [3];
2) изучение алгоритмов для генерации задач и работы с рисовкой фигур;
3) разработка локального MVP продукта для клиентов MacOS для решения поставленной цели;
4) наполнение базовой версии приложения дополнительным функционалом;
5) анализ экономической составляющей продукта.

1. ИССЛЕДОВАТЕЛЬСКАЯ ЧАСТЬ

1.1. Поле и поиск в глубину

Первым делом, нужно разработать концепцию геометрического поля, которая позволит пользователю решать поставленные задачи. Следует хранить информацию об отмеченных точках, нарисованных прямых и анализировать получившиеся геометрические фигуры.

Игра должна быть наполнена совершенно разными головоломками: от поиска способа соединения уже нарисованных точек до построения геометрической фигуры с определенным характеристиками.

Граф G = (V, E) – это множество вершин V и множество соединяющих их ребер E.

Если представить точки на поле в виде вершин графа, то прямые можно анализировать как ребра этого графа и хранить всё представление в списке смежности. Благодаря этой реализации ребра становится исследовать существенно удобнее.

Один из оптимальных методов обхода для нахождения пути или цикла определенной длины – поиск в глубину, также известный как DFS.

На рисунке 1 представлен граф G из пяти вершин. Для запуска поиска в глубину нужно выбрать конкретную вершину.

Начиная поиск в глубину из вершины D, сначала нужно пройтись по смежным ей вершинам. Допустим, алгоритм попал в E. Эта вершина еще не была отмечена, потому что она впервые встретилась в обходе. Кроме того, из нее выходят другие ребра. Следует отметить E и продолжить обход, теперь проверяя вершины, смежные уже этой E.

Вершина помечается, когда алгоритм впервые посещает ее. Повторно пройти по ней нельзя. Если обход достиг вершины, у которой нет смежных или они все уже помечены, то следует остановить рекурсию и вернуться на шаг назад.

Во время обхода можно добавлять дополнительные условия для остановки рекурсии, ограничивая длину пути, сохраняя маршруты или проверяя финальную точки пути. Это поможет в проверке количества углов многоугольника, его замкнутости и дальнейшем анализе выпуклости при необходимости.

Например, на графе с рисунка 1 можно найти циклы длиной 4: D – C – A – E – D или C – B – E – A – C.

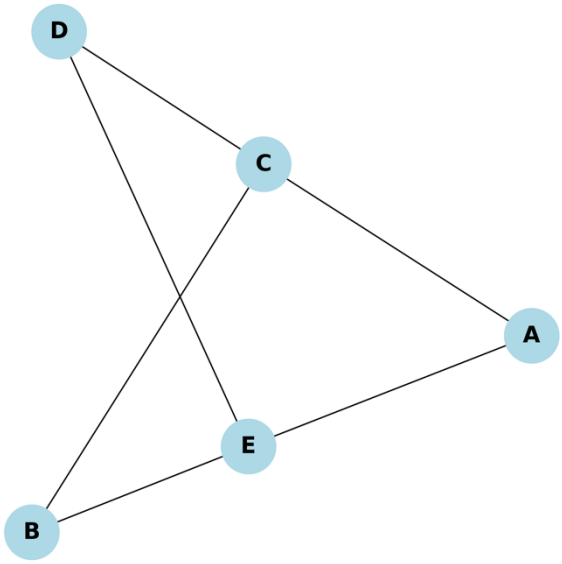

Рисунок 1 – Граф G и вершины A, B, C, D, E

Оценка алгоритма обхода в глубину – O(|V| + |E|), где |V| – мощность набора вершин, а |E| – мощность множества ребер. Если учитывать тот факт, что поле будет ограничено примерно 50 вершинами, а количество ребер также ограничено количеством разнообразных линий, то можно подтвердить, что прорисовка задач, поиск решений и проверка ответов будут занимать допустимое количество времени.

1.2. Генерация заданий и адаптивная сложность

Необходимо выбрать способ, согласно которому будет определяться, какое из конкретных заданий получит пользователь. Для этого следует проранжировать головоломки по уровню сложности. Любой пользователь при

регистрации получит начальный уровень сложности, чтобы сначала разобраться с простыми задачками и понять все нюансы управления.

Решая задачи, уровень игрока повышается с определенным параметром α, поэтому ему начинают попадаться более сложные задачи:

$$s := s + \alpha \tag{1}$$

Совершая ошибки, уровень будет уменьшаться, хоть и с меньшей скоростью и другим параметром β:

$$s := s - \beta, \beta < \alpha \tag{2}$$

Показатель умений пользователя будет ограничен минимальным и максимальным значениями сложности головоломок. Сами головоломки подбираются в соответствие с уровнем игрока, хоть и с небольшой погрешностью, то есть принадлежат установленному промежутку:

$$d \in [s - \Delta, s + \Delta] \tag{3}$$

Это упрощенная модель адаптивной сложности.

1.3. Аналитическая геометрия

Помимо поиска в глубину, для проверки решений будут использоваться и другие условия. Например, выпуклость многоугольника можно проверить через векторное произведение, если сохранить потенциальные пути-кандидаты после обхода в глубину и отсортировать точки по координатам их расположения и углу между векторами.

Векторное произведение позволяет проверить направление одного вектора относительно другого. Попарно перемножив все вектора фигуры, начиная и заканчивая левой крайней точкой, следует проверить знаки получившихся значений. Для каждого векторного произведения будем брать три точки с координатами (x1,y1), (x2,y2), (x3, y3) в порядке обхода по часовой стрелки или против:

$$z = (x_2 - x_1)(y_3 - y_1) - (y_2 - y_1)(x_3 - x_1) \tag{4}$$

Переменная z хранит в себе результат векторного произведение, который представляет из себя числовое значение.

Если все значения z в результате таких перемножений получатся с одинаковым знаком, то значит все векторы направлены в одинаковую сторону поворота по отношению к друг другу и образуют выпуклую фигуру.

Следует также использовать координаты для каждого объекта, чтобы удобнее работать с поиском середин отрезков и фигур, проверять параллельность прямых через пропорциональность уравнений и другие некоторые геометрические характеристики.

2. ПРАКТИЧЕСКАЯ ЧАСТЬ

2.1. Первоначальная отрисовка поля

На рисунке 2 представлена визуальная реализация поля и экрана заданий. Примерно 75% игрового экрана занимает именно поле. Линии проходят через все координаты, кратные определенному числу. В данном случае использовалась делимость на 100.

Узел – это место, где пересекаются вертикальные и горизонтальные прямые. В узлах можно отмечать точки или соединять их отрезками. Располагать начало и конец отрезков вне узлов нельзя.

На рисунке 2 изображены 56 узлов. Каждый узел – это вершина графа. Для реализации этой системы были созданы классы поля, экрана заданий и специальный класс для рисования на поле.

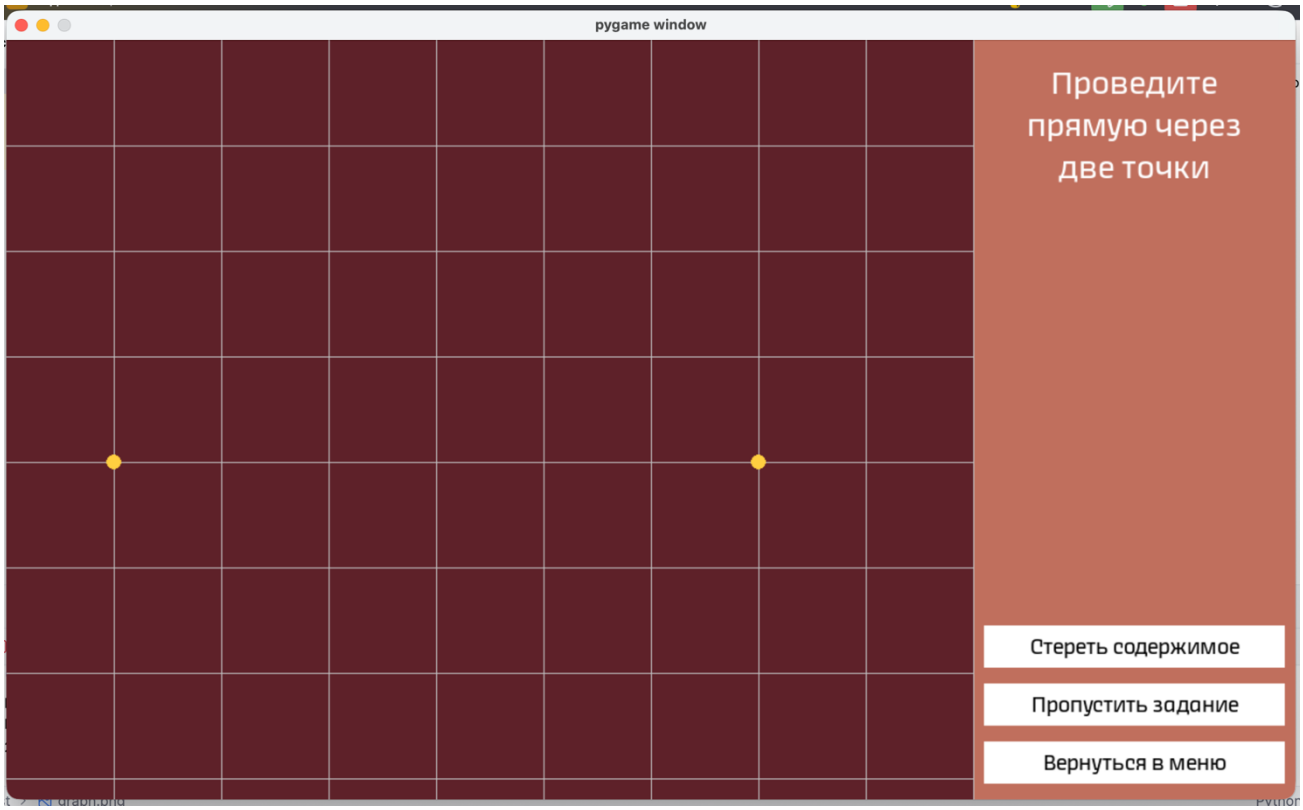

Рисунок 2 – Экран заданий

Когда пользователь впервые открывает вкладку заданий, в классе рисования создаются 56 точек с координатами узлов. Точки регулируются отдельным классом, в котором можно задать центр, радиус и цвет объекта.

Параллельно с этим процессом точки сохраняются в качестве ключей специального словаря. Этот словарь – атрибут класса рисования и представляет собой реализацию списков смежностей. Значение каждого ключа – это множество, изначально пустое. Впоследствии туда будут добавляться другие узлы-точки, которые соединены с вершиной через отрезки. Причем, необязательно, чтобы ключ был началом или конца отрезка. Код учитывает и промежуточные соединения, к чему вернемся чуть позже.

После того, как список смежности был создан, мы отдельно сохраним все узлы в матрицу размером 8 на 7, используя возможности библиотеки Numpy [1]. Это упростит проверку пользовательского ввода и отслеживание нажатий. Массив Numpy [1] также хранится в атрибутах класса для рисования поля.

2.2.1. Работа с полем. Очищение содержимого.

Во-первых, пользователь может стереть все содержимое поля (за исключением начальных условий задачи). Списки смежности очищаются полностью, а затем запускается двойной цикл, чтобы пройтись по координатам и достать из матрицы Numpy [1] сохраненные узлы. Они заново наполняют списки смежности как ключи с пустыми множествами.

2.2.2. Работа с полем. Добавление точки.

Чтобы добавить точку, достаточно нажать на любой из узлов. Для обработки нажатий на поле был создан отдельный класс со специальной функцией проверки.

Когда система получает информацию о нажатии, производится проверка, в какую именно область указал пользователь, какой обработчик должен быть активен и была ли зажата кнопка мышки до этого момента. Если игрок попал в поле и до этого клавиша мыши была поднята, то на поле рисуется точка.

Если после этого пользователь сразу поднимет кнопку, то точка будет зафиксирована.

Кроме того, система не требует нажатия в точные координаты центра узла. Существует погрешность, на которое можно отклониться от нужного пикселя. Отслеживается это все через специальную функцию, в которой проверяется, не вышел ли игрок за границы и попадает ли он в допустимые значения погрешности. Значение погрешности задается в зависимости от масштаба.

2.2.3. Работа с полем. Получение узла.

Стоит отметить, что после всех проверок в коде сохраняется кортеж из координат, а не непосредственно точка. Чтобы сохранить точку во временной переменной, используется массив Numpy [1]: координаты нужного объекта делятся на длину или ширину клетки в пикселях, что в нашем случае одно и то же. Чтобы узнать индексы элемента, из высчитанных значений еще необходимо вычесть единичку.

2.2.4. Работа с полем. Отрезки.

Если пользователь не отпустил нажатие и продолжает водить курсором в пределах границ поля, система проверяет, где конкретно находится мышка. Если курсор находится относительно недалеко от другого узла, то система автоматически наставляет его на точку. В противном случае, второй конец отрезка отмечаться не будет, как и показано на рисунке 3.

Временные отрезки не хранятся в списке смежности. Они отдельно представлены в атрибутах класса поля. Это сделано для того, чтобы проверка решения осуществлялась строго после окончательного выбора пользователя.

Если пользователь отпустил клавишу и не довел второй конец до узла, то тогда прямая исчезнет. В противоположной ситуации прямая будет обработана, чтобы все новые связи появились в списке смежности.

Сначала система отсортирует начало и конец отрезка. Затем, получив их индексы через координаты, отберет все элементы в том массиве Numpy [1], которые могут лежать на этой прямой.

Остается лишь проверить лежат ли эти три точки на одной прямой. Сделать это можно довольно просто, с помощью известного тождества. Нужно взять координаты и подставить их:

$$(x_3 - x_1)(y_2 - y_1) = (y_3 - y_1)(x_2 - x_1) \quad (1)$$

Если тождество выполняется, значит нам нужно сохранить точку (x_3, y_3) в отдельный список. После проверки всех точек из выбранного среза массива, запускается двойной цикл и аккуратно добавляются все связи. Сложность алгоритма в худшем случае– O(размер матрицы в высоту или ширину).

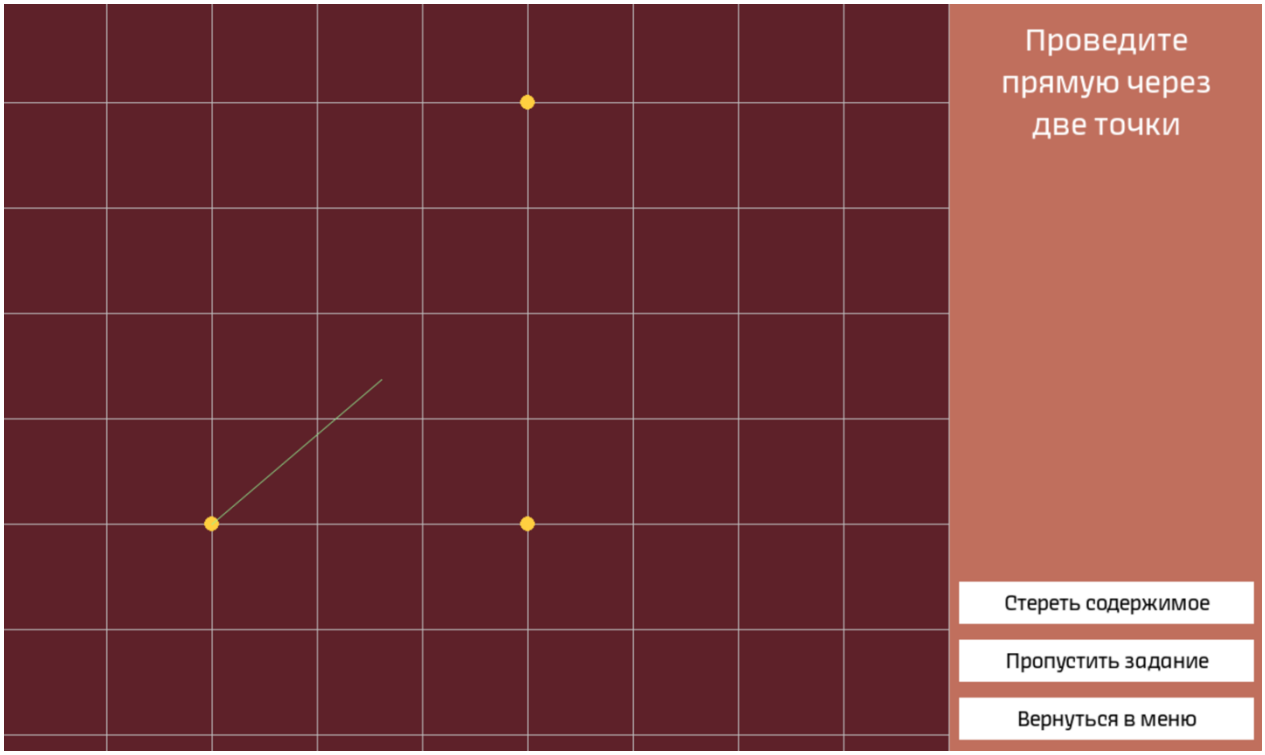

Рисунок 3 – Временная прямая

В классе для рисовки на поле, фактически, хранятся два списка смежности. Если первый представляет любые соединения, даже промежуточные, то во втором учитываются только цельные отрезки и точки в их концах. Это позволяет быстрее отрисовывать поле, чтобы не тратить отдельно времени на проверку нужных связей. Из-за размера поля небольшие затраты на память окупаются лучшей производительностью.

2.3.1. Работа с заданиями. База данных

Вся информация об игре хранится в базе данных формата SQLite [5].

На рисунке 4 представлена примерная схема архитектуры базы данных. Каждая запись в таблице с заданиями состоит из уникального идентификатора, шаблона начальных параметров, шаблона функции для проверки решения и уровня сложности.

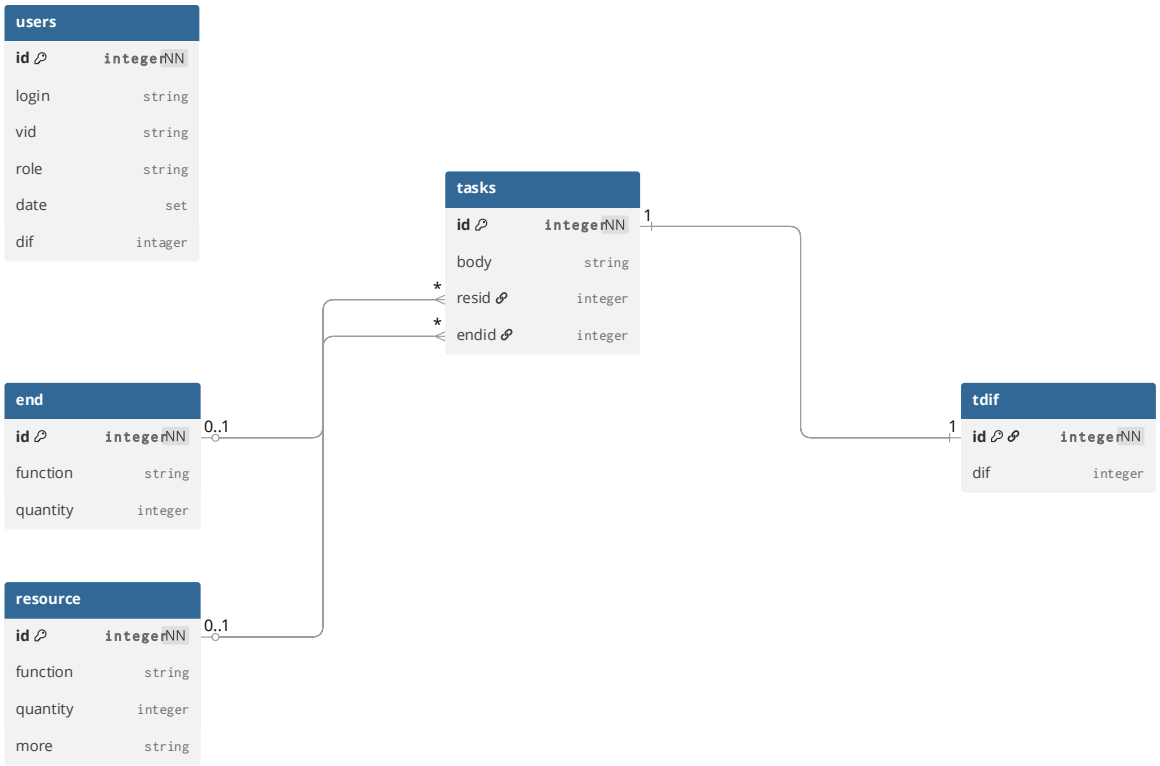

Рисунок 4 – Архитектура базы данных

В базе в шаблонах начальных условий и проверки решения хранятся названия функций и дополнительные параметры, большинство из которых – числовые условия. При загрузке поля или нажатии кнопки пропуска или перехода к следующему нажатию, в зависимости от уровня сложности пользователя на текущий момент, подгружается одно из заданий. В классе экрана с заданиями есть специальная функция, которая, когда загружает выбранное задание, записывает его текстовую часть в надпись в правую верхнюю часть экрана, сохраняет начальные объекты в отдельном списке в атрибуте класса поля и указывает метод класса проверки, который будет вызываться для сравнений требуемого ответа и нынешнего рисунка.

Проверка запускается в самом главном файле (классе Game) после того, как обработались новые нажатия пользователя, чтобы проверить его действия.

Таким образом, существует два класса: класс начальных условий и класс конечных условий. Мы создаем объект этих классов и сохраняем их в атрибуты Game. Затем мы по названию получаем значение метода объекта благодаря функции getatttr(). Теперь задаем все конечные условия объекту проверки заданий и сохраняем название нужной функции как объект.

2.3.2. Работа с заданиями. Генерация начальных условий

Существует несколько уже готовых функций, позволяющих создать начальное условий. Во-первых, можно отметить сколько угодно узлов. При этом система следит, чтобы отмеченные узлы не повторялись. В данном случае точки создаются поочередно, выбираются с использованием np.random.choice и, в целом, библиотеки random из допустимого набора значений.

Кроме того, активно используется метод, которая передает пустой список начальных объектов. К слову, сам список хранится как атрибут класса поля и рисуется сразу вместе с ним.

2.3.3. Работа с заданиями. Проверка. Прямые

Чтобы проверить наличие связей между двумя точками, достаточно лишь пройтись по спискам смежности и проверить взаимное наличие точек в множествах.

Если требуется, чтобы несколько узлов были соединены в одну ломаную или прямую, то для этого следует использовать обход в глубину.

В данном случае достаточно лишь проверить наличие путей определенной длины. Обход нужно поочередно запускать из точек, которые должны быть соединены. Сначала всегда следует проверять, не превышено ли значение этой длины. Если нет, то обход продолжается, иначе возвращается на шаг назад.

Переходим к новой вершине, временно сохраняем ее в список. Далее проверяем ее смежные. Если система нашла путь нужной длины, состоящий из нужных вершин, то этот путь сохраняется в атрибут класса проверки, а обход продолжается.

В противном случае сначала проверяется, что вершины нету в текущем пути, а также смотрится ее наличие в списке начальных условий. При успешном прохождении отбора, вершина остается в текущем списке, а рекурсия продолжается на следующем уровне. Иначе вершину удалят из нынешнего пути, а сама программа вернется на уровень назад.

2.3.3. Работа с заданиями. Проверка. Многоугольники.

Обход в глубину для поиска замкнутых фигур работает по похожему принципу. Однако он поочередно запускается от всех вершин поля, а не только от стартовых узлов. Помимо этого, нас интересует не просто путь определенной длины, а цикл.

Поэтому, в дополнение к предыдущим проверкам, при определенной длине появляется условие про наличие стартовой точки в списке смежности рассматриваемой. Это позволяет уменьшает итоговой список найденных объектов.

Итоговый список сохраняется в специальном списке в атрибутах класса для рисования на поле. С этим атрибутом впоследствии работает объект класса проверки. Он проходит по полученным результатам и сравнивает дополнительные критерии.

Если требуемый ответ будет найден, тогда экран заданий будет обновлен, что и представлено на рисунке 5. Кроме того, пользователь может пропустить текущее задание и сразу перейти к следующему.

Все пропуски и решенные задачи учитываются при подсчете текущего уровня пользователя сложности. Если пользователь выйдет в меню, не решив задачу, то задача не будет генерироваться повторно, пока игра не закрыта или игрок не выйдет из своего аккаунта.

Если пользователю не знакомы термины из условий, то он может воспользоваться базой знаний. Там визуально представлены все геометрические фигуры, которые могут потребоваться для поиска решения. Внешний вид этой вкладки представлен на рисунке 6.

Добавление подсказок не является целесообразным на данном уровне разработки продукта, так как задачки в любом случае будут повторяться, давать пользователю бесконечно их решать через подсказку – не лучшее решение.

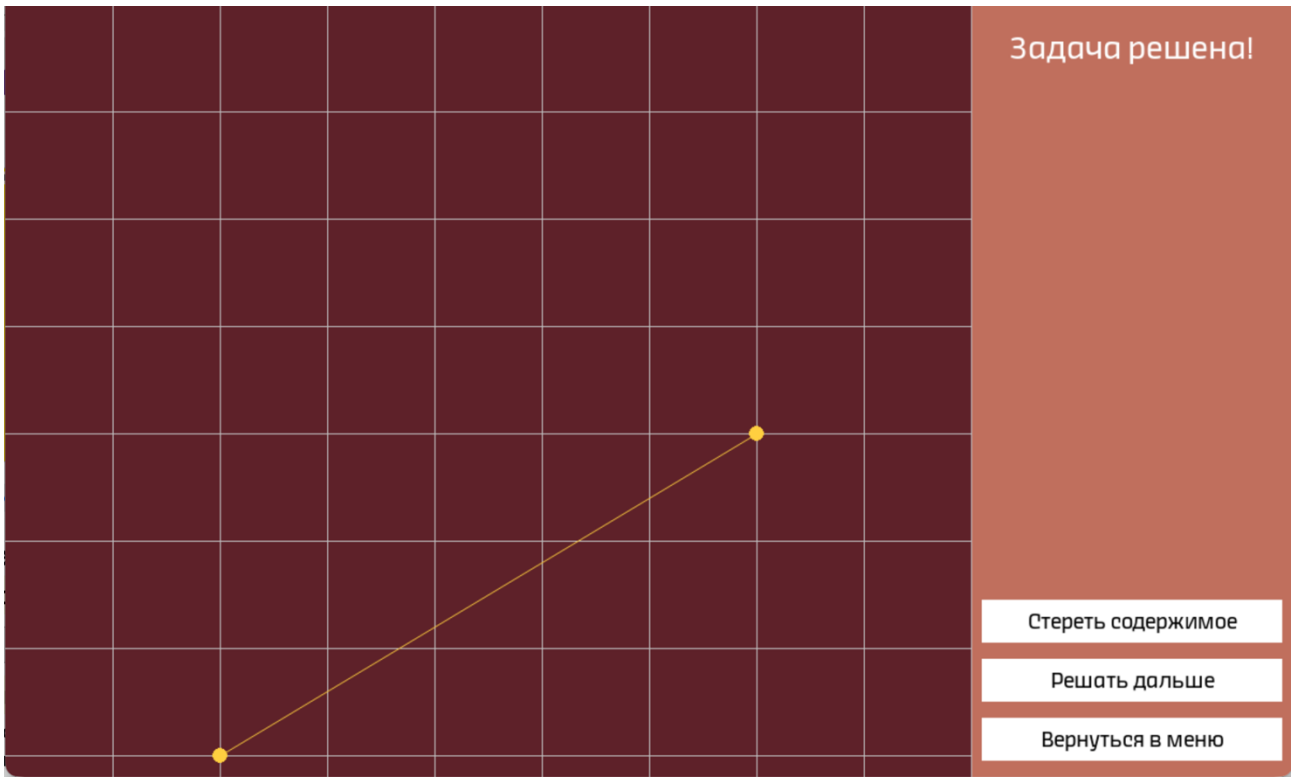

Рисунок 5 – Экран смены задания

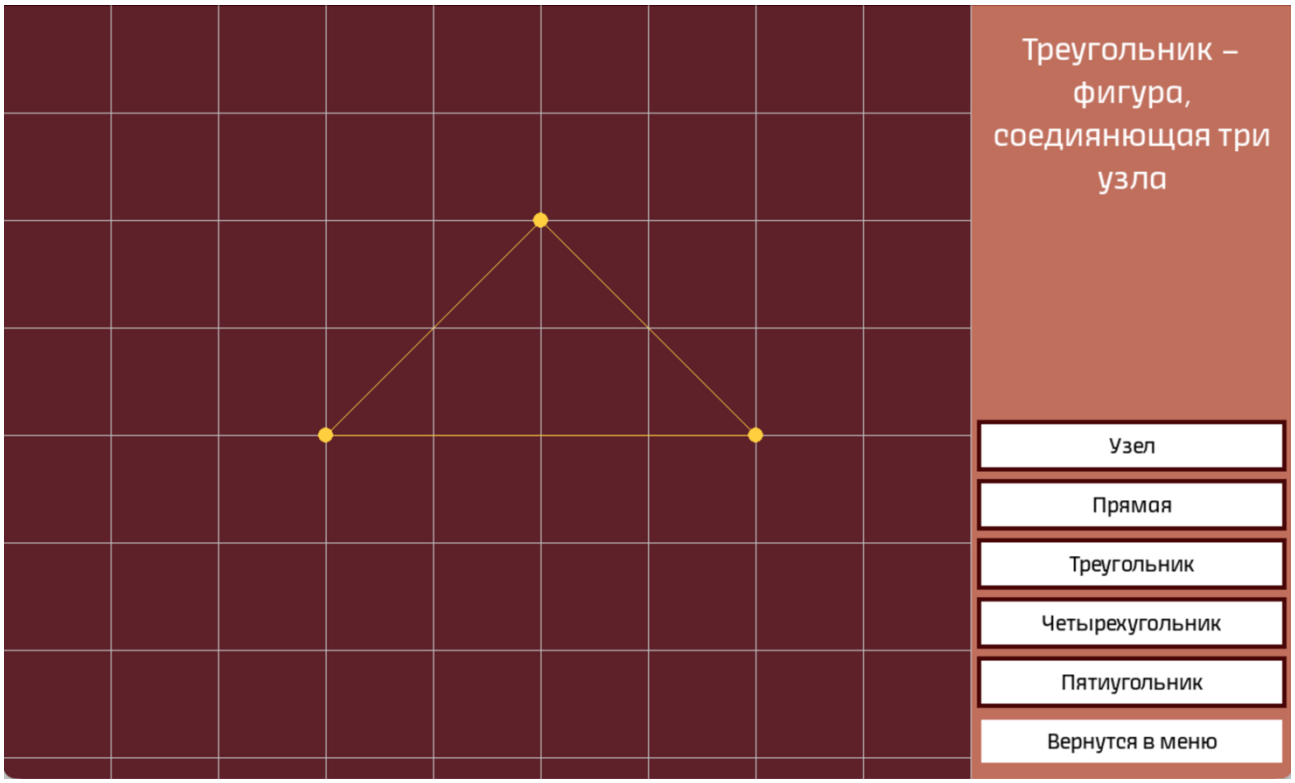

Рисунок 6 – база знаний

Дополнительные критерии для анализа тут связаны с геометрией.

2.3.4. Работа с заданиями. Проверка. Выпуклость

Например, в части заданий требуется нарисовать именно выпуклые фигуры. Чтобы проверить этот геометрический факт, пройдемся по всем циклам, найденным в результате обхода поля в глубину. Здесь требуются именно циклы, ведь выпуклость проверяется в замкнутых многоугольниках.

Сначала следует по координатам найти самую левую нижнюю точку. Сделать это можно с помощью функции min в Python [2], указав в качестве ключей сортировки – центр каждой из точек. Следующий шаг – вынести в отдельный список все точки, кроме стартовой.

Теперь воспользуемся функций сортировке. В качестве ключа используется угол между вектором от точки x до точки y и осью x. Он считается при помощи функции atan2 из библиотеки math. Она корректно работает при нулевых значении и связана с диапазоном от – π до π. Для ситуаций, когда несколько точек лежат на одной прямой, используется сортировка по второму параметру: расстоянию от самой нижней точки до изучаемого объекта.

Отсортировав точки, система считает векторное произведение для последовательных троек и сохраняет промежуточные значения. Затем эти промежуточные значения сравниваются для анализа, описанного в исследовательской части.

2.4.1. Адаптивная сложность. Подсчет сложности

Когда пользователь нажимает на кнопку перехода к следующей задаче или пропуска нынешней, система пересчитывает его сложность.

Прежде всего, из базы данных из таблицы TDIF достается уровень сложности задачи, номер которой сохранен в атрибуте объекта класса пользователя.

Если пользователь верно решил головоломку, то он получает очки по формуле:

$$s = s + 0.3 + tdif * 0.05 \quad (2)$$

Текущие очки (s) хранятся в одном из атрибутов класса User.

Если пользователь прошел авторизацию, то основной класс (Game) создает объект класса User со всеми нужными характеристиками. Во время начала сессии каждому пользователю присваивается значение в 0 очков.

Уровень сложности задачи влияет на скорость накопления очков пользователя. В случае негативного кейса, а именно пропуска головоломки, очки пользователя незначительно падает. В данном варианте штрафной коэффициент значительно больше:

$$s = s - 0.3 - tdif * 0.1 \quad (3)$$

Если очки игрока превысили значение в 1 единицу, то его уровень сложности повысится на один пункт. Если же количество баллов станет меньше, чем -1, то уровень сложности пользователя уменьшится на 0.5. После обновления уровня сложности, количество очков приравнивается к 0.2.

Когда пользователь будет выходить из аккаунта или игры, система округлит его уровень сложности через встроенные функции Python [2] и сохранит полученное значение в специальной колонке таблицы User.

2.5.1. Особенности интерфейса

Прототип игры был написана на языке Python [2] с использованием только стандартной библиотеки Pygame [3]. В ней отсутствуют некоторые базовые компоненты интерфейса, такие как кнопки или поля ввода.

Это важные объекты, поэтому для стабильной работы приложения были написаны классы Button и Entry соотвественно. Их идея достаточна проста - за основу наследования берется класс pygame.Rect.

Каждой кнопке можно присвоить свою функцию, отслеживать действия при нажатии и задавать минимальную стилизацию с помощью методов классы. Аналогичные функции подготовлены для Entry.

Кроме того, в случае поля ввода, у пользователя есть возможность задать, какие символы следует ожидать в качестве ввода и нужно ли скрывать введенную информацию.

2.5.1. Реализация информационных сообщений

Отдельно стоит упомянуть реализацию информационных системных сообщений. Благодаря информации на портале Stack Overflow [4] и в соответствующих поисковых запросов для воссоздания этого функционала была использована библиотека subprocces и команда osascript, доступная на устройствах с операционной системой MacOS.

3. ЭКОНОМИЧЕСКОЕ ОБОСНОВАНИЕ

3.1. Исследование потребителя

Центральная аудитория продукта – это люди в возрасте с 8 до 70 лет. Такие возрастные ограничения связаны с тем, что более старшее поколение не так активно пользуется мобильными устройствами. Хотя, конечно, все ситуации индивидуальны. Игра будет доступна и для серебряного возраста, ведь ее правила довольно просты и не требуют больших познаний. Это отличный способ потренировать мозговую активность по аналогии с кроссвордами и другими интересными головоломки.

Подводя итог, универсальность и простота задумки позволят успешно предложить продукт нескольким возрастным категориям.

3.2 Анализ конкурентов. Horis International Limited.

И Пифагорию, и Euclidea курирует один и тот же издатель. У этого издателя есть и другие игры. Впринципе, их концепция похожа: есть какая-то конкретная тематика, которой посвящен большой набор задач. Это издатель выпускал даже игру XSection, основанную на изучении трехмерного пространства.

К сожалению, в интернете практически не найти информацию об этой фирме. Остается доступен лишь сайт, который, судя по внешнем виду, не обновлялся несколько лет. Найти информацию по названию на сайтах регуляторах различных стран не удается. Кроме того, в школе я слышал о том, что над этой игрой работали жители Санкт-Петербурга.

У игры Euclidea есть паблик в социальной сети ВКонтакте. Там до сих пор довольно активно публикуются еженедельные посты, за которыми следят минимум три тысячи человек. Кроме того, во всемирной паутине можно найти множество материалов, посвященных решению различных задач. Соответственно, даже если и минимальная, но аудитория преданных фанатов у этой серии существует, а новые пользователи продолжают скачивать игры.

Судя по всему, компания не обладает большим штатом разработчики, раз не декларирует публично свои отчеты. В самих приложениях практически отсутствует монетизация. Вы лишь можете поддержать их донатом.

Профили, привязанные к социальным сетям игр, также не раскрывают большое количество информации об экономической составляющей издателя, как и любых других подробностей.

Резюмируя небольшой анализ, можно увидеть возможность для выхода на рынок и конкуренции с большим игроком благодаря различиям в основных концепциях.

3.3.1. Затраты. Единовременные

Основные единовременные затраты связаны со сферой маркетинга. Сначала нужно провести консультации с профессионалами в этой сфере, разработать правильную стратегию рекламы. На это следует выделить 20 000 рублей.

Так что единоразово нужно выделить бюджет на закупу рекламы, баннеров и билбордов: 100 тысяч рублей. Лучше дополнительно охарактеризовать эту сумму после консультаций с маркетологами. Планируется закупка рекламы в Telegram, работа с инструментами Яндекса и метрополитена города Санкт-Петербург.

Отдельно стоить выделить 35 тысяч на покупку фирменного брендбука и разработку рекламных плакатов у профессиональных дизайнеров.

Затем можно лишь поддерживать игру в информационным поле с помощью рекламных инструментов Яндекса, о чем было упомянуто в ежемесячных затратах.

Возможно, в будущем для расширения базы головоломок игры потребуется нанять специальный персонал. Но это слишком далекие планы, поэтому учитывать эти затраты смысла нету.

Все затраты помогут успешному запуску продукту и будут окуплены в будущем.

3.3.2. Затраты. Электроэнергия

Все оборудование уже имеется у меня в наличии, поэтому тратить деньги на дополнительные закупки не стоит.

Чтобы посчитать затраты на электроэнергию, нужно воспользоваться следующей формулой:

$$Э = Ц * (Т * Р) \quad (1)$$

Итак, Т – это время. В среднем, за компьютером я буду проводить по 7 часов, чему и равно Т. Р – мощность компьютера, в данном случае 100 Вт/час. Получается, что в среднем я потрачу 15 500 Вт (15 кВт), если буду работать 22 дня в месяц с двумя выходными каждую неделю.

Это значение попадает в первый диапазон объемов мощности, поэтому ставка составит 6.97, а общая сумма 15 * 6.97 ≈ 110 рублей в месяц. Я буду специально работать в то время, когда комнату озаряет естественное освещение.

3.3.3. Затраты. Ежемесячные

Из основных ежемесячных затрат, которые потребуются для поддержания жизненного цикла проекта, следует выделить:

1) затраты на сервера игры и хранение базы данных – пока проект лишь ищет свою аудиторию и не особо популярен, можно арендовать сервер за 10 тысяч в месяц, при необходимости увеличивать количество ресурсов
2) плата за публикацию проекта в магазинах приложений – примерно 8000 рублей в месяц
3) бухгалтерский и юридический аутсорс для заполнения всей отчетности – примерно 5000 рублей в месяц
4) затраты на поддержание игры в информационным поле – примерно 15 000 рублей в месяц

Все цены собраны после опроса потенциальных кандидатов, с которыми уже были наработаны связи и успешное сотрудничество.

Будучи основным разработчиком и единственным штатным сотрудником, я бы хотел иметь некий постоянный оклад. Тем не менее, я понимаю, что без успешного привлечения аудитории этого не получится, поэтому готов инвестировать большие средства вначале или найти достойных инвесторов и получить у них финансовую поддержку до становления сильным игроком рынка.

Пока что заключать договор со специальной компанией технической поддержкой не требуется, ибо я могу сам отвечать на вопросы по электронной почте, так как занимаюсь только этим проектом. Кроме того, я уже имел опыт в работе в качестве технической поддержки, так что я быстро смогу влиться в процесс.

Несмотря на значительный размер ежемесячных затрат, все они связаны с довольно важными организационными моментами и стабильным функционалом, там что не следует пренебрегать этими пунктами и постепенно заниматься оптимизацией ресурсов.

3.3.4. Затраты. Подведение итогов

Если объединить все ежемесячные затраты и добавить туда мою оплату труда по рыночной цене, примерно 120 тысяч, а также всякие технические расходы, подсчитанные бухгалтерами, получится примерно 170 тысяч в месяц. Сюда включены даже амортизационные отчисления. Задача – подобрать систему монетизации, чтобы получить больший доход и выйти на окупаемость.

3.4. Монетизация

Планировать делать приложение платным или ограничивать критически важный функционал при наличии бесплатных конкурентов – проигрышная стратегия. Следует добавить в игру нативную рекламу, чтобы выйти хоть на какую-то окупаемость. Конечно, спрогнозировать этот доход тяжело. Однако в случае достижения большой аудитории, этот источник дохода может приносить большие суммы.

Кроме того, нужно подумать о возможности кастомизации интерфейса и объектов с помощью продажи различных вариаций внешнего интерфейса. Даже установив довольно маленькую цену на покупку, можно найти заинтересованных покупателей.

В связи с тем, что игра завязана на одном человеке, следует учитывать все риски и возможные последствия этой авантюрой. Однако проект имеет интересную концепцию, решает важную проблему и не сложен в реализации, поэтому я уверен в его успехе.

3.5. Продажа лицензии

Продажа основной идеи – не самый лучший вариант достижения концепция. В этом случае я откажусь от участия в разработке проекта, забрав полученные деньги.

С учетом полученного размера бюджета и плавающего размера зарплат, я оцениваю свой проект в 100 тысяч рублей.

Это сумма связана с количеством времени, которое было затрачено мной для создания MVP. Оно соответствует примерно трем с половиной рабочих недель.

Если считать рыночной зарплатой 120 тысяч рублей в месяц, то за такой неполный месяц работы следует попросить чуть меньшую сумму.

ВЫВОДЫ

В ходе работы над продуктом я несколько раз изменял свое виденье концепции. Кроме того, изначального я планировал в отдельном разделе добавить пару задачек, связанных с функциями. Можно было бы находить параметры линейных и параболических функций, а также поискать другие интересные примеры.

Однако в процессе пришлось отказаться от этой идеи и сосредоточиться над разработкой геометрического режима. С этой задачей я справился успешно и очень доволен получившемся результатом.

У меня вызывает много проблемы работа над внешним видов собственных проектов. Здесь повезло, что я смог подобрать нужное сочетание цветов.

На моем пути возникали и другие проблемы: я долго думал над реализацией концепции поля, пару раз переписывал основной алгоритм работы с узлами, прямыми. Много раз менялась архитектура базы данных.

Еще один важный объект, который у меня реализовать не получилось – это администраторская панель. В настоящий момент нет унифицированного метода добавления задания. Я один работаю над игрой, поэтому необходимости в этом объекте пока нету.

Сейчас в игре представлено лишь несколько задач, поэтому следует поработать над более разнообразным контентом. Кроме того, нужно добавить фоновую музыку, увеличить количество настраиваемых параметров и подумать над поорщением пользователя за правильные ответы для дальнейшей мотивации отказа от скроллинга социальных сетей.

Я уверен, что моя реализация идеи отлично справляется с постановленными задачами и готова преобразиться в успешный полноценный коммерческий проект.

ЗАКЛЮЧЕНИЕ

В ходе проделанной работы были усовершенствованы навыки программирования на языке Python [2] с использованием библиотеки Pygame [3], изучены алгоритмы обхода графов и адаптивной сложности, улучшены знания о работе с SQLite [5], разработан локальный MVP продукта для устройств с операционной системой Mac OS.

Кроме того, проведенный экономический анализ помог определить дальнейшие перспективы развития продукты и цели на ближайшее время.

Я провел небольшой опрос целевой аудитории среди своих однокурсников. Они высказали заинтересованность в продукте, так что проблема с полезным препровождением в электронных устройствах тоже постепенно будет решена, уже были сделаны первые шаги.

Рисование фигур на поле работает стабильно, поэтому в проект можно добавлять дополнительные задачи, работать над разнообразием контента и постепенно повышением уровня сложности базы задач.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Numpy 2.3.5 [Электронный ресурс]. – США: Numpy Developers, 2025. – Режим доступа: https://numpy.org/, свободный. – Загл. с экрана. – Яз. англ.

2. Python 3.11 [Электронный ресурс]. – США: Python Software Foundation, 2025. – Режим доступа: https://www.python.org/, свободный. – Загл. с экрана. – Яз. англ.

3. Pygame 2.6.1 [Электронный ресурс]. – США: Pygame Community, 2025. – Режим доступа: https://www.pygame.org/news, свободный. – Загл. с экрана. – Яз. англ.

4. Stack Overflow: Programmer's Q&A Community [Электронный ресурс]. – Нью-Йорк: Stack Exchange Inc., 2025. – Режим доступа: https://stackoverflow.com/, свободный. – Загл. с экрана. – Яз. англ.

5. SQLite: 3.47 [Электронный ресурс]. – США, CT: SQLite Consortium, 2025. – Режим доступа: https://www.sqlite.org/, свободный. – Загл. с экрана. – Яз. англ.